

OpenMower-TAF-ROS

Änderungsdokumentation

GPS-State Standardisierung

Version v0.1 - 2026-07-13

Angabe	Wert
Dokumententitel	OpenMower-TAF-ROS - Änderungsdokumentation - GPS-State Standardisierung - v0.1
Projektname	OpenMower-TAF-ROS
Dokumentenart	Änderungs- und Implementierungsdokumentation / Codepaket
Thema	Einheitliche Definition-/Status-Topics, zentraler Renew-Befehl und Debug-Export-Vorbereitung
Erstellungsdatum / Änderungsdatum	2026-07-13
Version / Status	v0.1 / technisch umgesetzter Arbeitsstand
Verantwortlich	OpenAI GPT-5.5 Thinking - technische Ausarbeitung für OpenMower-TAF-ROS
Ablageort	_Dokumentation/ im Codepaket; PDF und DOCX zusätzlich als Auslieferungsdateien
Referenzen	OpenMower-TAF-ROS(5).zip; bestehende GPS-State-v2-Implementierung; Vorgaben zur Projektdokumentation

Wichtiger Umfangshinweis: Das ROS-/MQTT-Backend wurde im gelieferten Paket umgesetzt. Der gewünschte Download-Button in der Flutter-GPS-Seite konnte im aktuellen ROS-ZIP nicht kompiliert werden, weil dort nur der erzeugte Web-Build (<web/main.dart.js>), aber kein Flutter-/Dart-Quellcode enthalten ist.

1. Kurzfassung

Die GPS-State-MQTT-Schnittstelle wurde für State 0 bis State 4 auf eine einheitliche Struktur gebracht. Jeder State besitzt nun ein retained Definition-Topic und ein separates Status-Topic. Die manuelle Aktualisierung erfolgt zentral. Die fachliche Aufteilung der States bleibt unverändert.

- Kanonische Topics: `gps_state/stateN/definition` und `gps_state/stateN/status` für $N = 0$ bis 4.
- Zentraler Aktualisierungsbefehl: `gps_state/set/renew/json`.
- Gemeinsames Payload-Schema `gps_state.v3` mit Definition-Version 3.
- Gemeinsamer Statusrahmen mit Veröffentlichungszeit, Quellzeit, Alter, Verfügbarkeit, State-Kennzeichen, Status, Schweregrad, Zusammenfassung und data-Block.
- State 1 bis State 4 erhalten erstmals eigene statische Definitionen.
- Definitionen werden nicht mehr bei jedem regulären 1-Hz-Zyklus wiederholt.
- Bisherige State-1-bis-State-4-Topics und das frühere State-0-Renew-Topic bleiben vorübergehend als Kompatibilitätsalias erhalten.

2. Ausgangslage und Ziel

Vor der Änderung war nur State 0 konsequent in Definition und Status getrennt. State 1 bis State 4 wurden als einzelne Payloads auf `gps_state/state1` bis `gps_state/state4` veröffentlicht. Zusätzlich existierte eine nur für State 0 gültige Aktualisierungsanforderung. Dadurch musste die App pro State unterschiedliche MQTT- und Aktualisierungslogik verwenden.

Bereich	Bisher	Neu
Topic-Struktur	State 0 getrennt; State 1-4 jeweils ein Topic	Alle States: definition + status
Manuelle Aktualisierung	<code>gps_state/state0/set/renew/json</code> nur für State 0	<code>gps_state/set/renew/json</code> zentral, mit State-/Part-Auswahl
Statische Metadaten	Nur State 0 vollständig definiert	Definitionen für State 0-4
Statusrahmen	Unterschiedliche Zeit- und Statusfelder	Gemeinsamer <code>gps_state.v3</code> -Rahmen
Migration	Bestehende App erwartet alte Topics	Alte Status-Topics werden als Alias weitergesendet

3. Zielstruktur der MQTT-Topics

State	Definition (retained)	Status	Retain Status	Rolle
0	<code>gps_state/state0/definition</code>	<code>gps_state/state0/status</code>	Ja	12-stufige Fahrfähigkeitsdiagnose
1	<code>gps_state/state1/definition</code>	<code>gps_state/state1/status</code>	Ja	Kompakter Bedienerstatus
2	<code>gps_state/state2/definition</code>	<code>gps_state/state2/status</code>	Ja	Technische GNSS-/Pose-Zusammenfassung
3	<code>gps_state/state3/definition</code>	<code>gps_state/state3/status</code>	Nein	Aktuell verwendete Satelliten
4	<code>gps_state/state4/definition</code>	<code>gps_state/state4/status</code>	Nein	Alle sichtbaren Satelliten / Debug

State 3 und State 4 bleiben nicht retained, weil Satellitenlisten Momentaufnahmen sind. Eine alte retained Liste könnte nach einem Neustart irrtümlich als aktuelle Messung erscheinen.

4. Einheitlicher Definitionsstandard

Jede Definition besitzt denselben äußeren Rahmen. Der state-spezifische Inhalt steht in fields; State 0 ergänzt weiterhin checks für die zwölf Entscheidungsknoten.

```
{
  "schema": "gps_state.v3",
  "state": "state1",
  "type": "definition",
  "definition_version": 3,
  "name": "gps_operator_status",
  "label": "GPS-Fahrstatus",
  "description": "Kompakter Bedienerstatus ...",
  "role": "operator",
```

```

"order": 10,
"status_retained": true,
"update_mode": "periodic_and_event",
"fields": { ... }
}

```

Gemeinsames Feld	Bedeutung
schema	Version des GPS-State-Datenmodells.
state	state0 bis state4.
type	Immer definition.
definition_version	Version der statischen Felddefinition.
name / label / description	Technischer Name und direkt darstellbare Texte.
role / order	Zielgruppe und Reihenfolge in der Oberfläche.
status_retained	Beschreibt das Retain-Verhalten des zugehörigen Status.
update_mode	periodic_and_event oder sensor_event.
fields	Statische Beschreibungen, Typen, Einheiten und Reihenfolge der Statusfelder.

5. Einheitlicher Statusstandard

Alle dynamischen Meldungen werden vor der Veröffentlichung in einen gemeinsamen Rahmen überführt. Die bisherigen state-spezifischen Felder bleiben zur Kompatibilität auf oberster Ebene erhalten und werden zusätzlich unter data gruppiert.

```

{
  "schema": "gps_state.v3",
  "state": "state1",
  "type": "status",
  "definition_version": 3,
  "published_at": 1783940400.25,
  "source_at": 1783940400.10,
  "age_ms": 150,
  "available": true,
  "stale": false,
  "status": "ok",
  "severity": 0,
  "summary": "GPS reicht zum Fahren aus",
  "data": { ... }
}

```

Feld	Bedeutung
published_at	Zeitpunkt der MQTT-Erzeugung.
source_at	Zeitpunkt der maßgeblichen Eingangsdaten.
age_ms	Alter der Quelle beim Veröffentlichen.
available	State-Datenquelle ist grundsätzlich verfügbar.
stale	Quelle ist älter als die gemeinsame 3-Sekunden-Schwelle.
status	ok, warning, blocked, stop, unavailable oder unknown.
severity	0 = OK, 1 = unbekannt, 2 = Warnung, 3 = blockiert, 4 = Stopp.
summary	Kurzer direkt anzeigbarer Text.
data	State-spezifischer Datenblock; Top-Level-Felder bleiben in der Übergangsphase erhalten.

6. Zentrale Aktualisierung

Die manuelle Datenanforderung erfolgt über ein einziges Topic:

```
gps_state/set/renew/json
```

Leerer Inhalt oder {} aktualisiert Definition und Status aller aktivierten States. Eine explizite State-Auswahl wird unabhängig von den regulären Publish-Schaltern beantwortet.

```

{
  "states": [0, 2],
  "parts": ["status"]
}
{
  "states": [0, 1, 2, 3, 4],
  "parts": ["definition"]
}

```

- states akzeptiert 0 bis 4, Zeichenketten wie state2 sowie all.

- parts akzeptiert definition und status.
- Ungültige Payloads werden abgewiesen und über `gps_state/settings/validation/json` gemeldet.
- Die Anfrage löst keine neue physische F9P-Messung aus, sondern bewertet die jüngsten ROS-Eingänge.
- Fehlt noch ein Satelliten-Snapshot, werden definierte unavailable-/Leerstatus-Payloads erzeugt.

7. Veröffentlichungs- und Kompatibilitätsverhalten

7.1 Statische Definitionen

- Beim Aufbau der MQTT-Verbindung.
- Bei `gps_state/settings/set/renew/json`.
- Nach einer Änderung der GPS-State-Einstellungen.
- Bei einer zentralen Renew-Anforderung mit `part=definition`.

Die Definitionen werden bewusst nicht mehr in jedem regulären Statuszyklus erneut gesendet.

7.2 Dynamische Statusmeldungen

Reguläre Statusmeldungen folgen weiterhin `publish_rate_hz` (Standard 1 Hz, einstellbar von 0,1 bis 5 Hz). Änderungen des GNSS-/RTK-Fixstatus können zusätzliche ereignisgesteuerte Meldungen auslösen.

7.3 Übergang für bestehende App-Versionen

Kanonisches Topic	Kompatibilitätsalias	Hinweis
<code>gps_state/state1/status</code>	<code>gps_state/state1</code>	vorübergehend parallel
<code>gps_state/state2/status</code>	<code>gps_state/state2</code>	vorübergehend parallel
<code>gps_state/state3/status</code>	<code>gps_state/state3</code>	vorübergehend parallel
<code>gps_state/state4/status</code>	<code>gps_state/state4</code>	vorübergehend parallel
<code>gps_state/set/renew/json</code>	<code>gps_state/state0/set/renew/json</code>	Alias aktualisiert nur State 0

Die Aliase verhindern, dass der im ROS-Paket enthaltene bereits kompilierte Web-Build sofort seine State-1-bis-State-4-Anzeige verliert. Neue App-Versionen sollen ausschließlich die kanonischen Topics verwenden.

8. Umgesetzte Codeänderungen

Datei	Änderung
<code>src/lib/xbot_monitoring/src/xbot_monitoring.cpp</code>	Neue Definitionen State 1-4; <code>gps_state.v3</code> -Statusrahmen; zentrale Renew-Auswertung; kanonische Status-Topics; Alias-Publishing; Definition-Publish-Lebenszyklus; Settings-Metadaten.
<code>src/lib/xbot_monitoring/README.md</code>	Neue Topicstruktur, Statusfelder, zentrale Aktualisierung, Retain- und Migrationsverhalten dokumentiert.
<code>_Dokumentation/...v0.1.docx / .pdf</code>	Bearbeitbare und stabile Änderungsdokumentation.
<code>_Dokumentation/...v0.1.patch</code>	Anwendbarer Text-Patch für die beiden Code-/README-Dateien.
<code>_Dokumentation/...Commit-Nachricht...txt</code>	Englische Commit-Nachricht mit BL_-Titel, Datum und vollständiger Dateiliste.

9. App-JSON-Download

9.1 Gewünschtes Verhalten

Die GPS-Seite soll analog zur Flächen-Seite eine einklappbare JSON-Ansicht mit direktem Download-Button erhalten. Der Export soll Settings sowie Definition und Status aller fünf States inklusive Topic- und Empfangszeit enthalten. Upload, Entsperren und Speichern sind für die reine GPS-Diagnose nicht vorgesehen.

JSON-Ansicht [Download] [↘]
GPS-Diagnosedaten anzeigen und herunterladen

9.2 Warum dieser Teil nicht in den Web-Build eingearbeitet wurde

Das bereitgestellte aktuelle Paket ist ein ROS-Projekt. Es enthält unter `web/` nur den bereits kompilierten Flutter-Web-Build. Die benötigten Dart-Dateien wie `lib/screens/gps_state.dart`, `lib/controllers/gps_state_controller.dart` und `lib/io/mqtt_connection.dart` sind nicht enthalten. Eine direkte manuelle Änderung von `web/main.dart.js` wäre nicht reproduzierbar und technisch nicht sicher.

Folge: Die MQTT-Backendänderung ist vollständig im Paket enthalten. Für den sichtbaren JSON-Download-Button wird zusätzlich das aktuelle OpenMower-TAF-App-Quellpaket benötigt. Der bestehende Web-Build bleibt durch die Kompatibilitätsaliases weiterhin nutzbar.

9.3 Vorgesehene App-Anbindung

- State 0 bis State 4 jeweils getrennt als definition und status speichern.
- Aktualisieren über `gps_state/set/renew/json`; keine dezentralen Renew-Topics in der neuen App verwenden.
- JSON-Ansicht als read-only Bereich unterhalb der GPS-Anzeige.
- Download über den bereits in der App vorhandenen plattformabhängigen Textdatei-Service.
- Dateiname: `openmower-gps-state-debug-YYYY-MM-DD_HH-mm-ss.json`.
- Export des aktuell in der App vorhandenen Snapshots; Aktualisieren und Download bleiben getrennte Aktionen.

10. Prüfung und Qualitätssicherung

Prüfung	Ergebnis	Bemerkung
C++-Syntaxbaum	Bestanden	Gesamte geänderte <code>xbot_monitoring.cpp</code> mit <code>tree-sitter-cpp</code> geparkt; keine ERROR-/Missing-Knoten.
JSON-Schema-Initialisierung	Bestanden	Isolierter C++17-Test mit <code>nlohmann::ordered_json</code> kompiliert und ausgeführt.
State-Definitionen 1-4	Bestanden	Isolierter C++17-Test erzeugte alle Definitionen und prüfte <code>type=definition</code> .
Status-Normalisierung	Bestanden	Isolierter C++17-Test prüfte <code>gps_state.v3</code> , <code>data</code> , <code>Status</code> , <code>Stale</code> - und <code>State-1</code> -Zeitverhalten.
Renew-Parser	Bestanden	Isolierter C++17-Test prüfte Standardauswahl, explizite States/Parts und Abweisung leerer Parts.
Topic-Vollständigkeit	Bestanden	Definition-/Status-Topics State 0-4 sowie zentraler Renew-Befehl statisch geprüft.
Vollständiger ROS-/catkin-Build	Nicht ausgeführt	ROS-Noetic-/Catkin-Toolchain ist in der Ausführungsumgebung nicht installiert.
Flutter-/Web-Build	Nicht ausführbar	Flutter-Quellcode und Flutter-SDK fehlen im bereitgestellten ROS-Paket beziehungsweise in der Ausführungsumgebung.

11. Empfohlene Inbetriebnahmeprüfung

1. ROS-Paket im normalen OpenMower-Build beziehungsweise CI kompilieren.
2. Mit `mosquitto_sub -v -t 'gps_state/#'` prüfen, dass alle fünf Definitionen retained eintreffen.
3. Leere Nachricht an `gps_state/set/renew/json` senden und die aktivierten State-Statusmeldungen prüfen.
4. Explizit `{"states":[0,4],"parts":["status"]}` senden und nur die angeforderten Statusantworten kontrollieren.
5. State 3 und State 4 nach Broker-Neustart auf nicht-retained Verhalten prüfen.
6. Bestehende Web-App auf den Legacy-Topics testen; anschließend neue App auf die `/status`-Topics umstellen.
7. Nach Bereitstellung des App-Quellpakets JSON-Ansicht und Download ergänzen und einen Flutter-Web-Build durchführen.

12. MQTT-Testbeispiele

```

mosquitto_sub -v -t 'gps_state/#'
mosquitto_pub -t 'gps_state/set/renew/json' -m '{}'
mosquitto_pub -t 'gps_state/set/renew/json' -m '{"states":[0,2],"parts":["status"]}'
mosquitto_pub -t 'gps_state/set/renew/json' -m '{"states":[0,1,2,3,4],"parts":["definition"]}'

```

13. Änderungshistorie

Version	Datum	Bearbeiter	Status	Änderung
v0.1	2026-07-13	OpenAI GPT-5.5 Thinking	Technisch umgesetzt	Einheitliche State-Topics, gps_state.v3, zentraler Renew-Befehl, Definitionen State 1-4, Kompatibilitätsaliases und Dokumentation.

14. Commit-Nachricht

BL_Standardize GPS state MQTT topics

Date: 2026-07-13

Summary:

- Standardized GPS State 0 through State 4 with retained definition topics and canonical status topics.
- Added the central gps_state/set/renew/json refresh command.
- Kept deprecated aliases for controlled client migration.
- Updated settings metadata and documentation.

15. Auslieferungsumfang

Artefakt	Dateiname
Vollständiges Projektpaket	OpenMower-TAF-ROS_GPS-State-Standardisierung_v0.1.zip
Geänderte Dateien	OpenMower-TAF-ROS_GPS-State-Standardisierung_changed-files_v0.1.zip
Patch	2026-07-13 - OpenMower-TAF-ROS - Patch - GPS-State Standardisierung - v0.1.patch
Commit-Nachricht	2026-07-13 - OpenMower-TAF-ROS - Commit-Nachricht - GPS-State Standardisierung - v0.1.txt
PDF-Dokumentation	2026-07-13 - OpenMower-TAF-ROS - Änderungsdocumentation - GPS-State Standardisierung - v0.1.pdf
DOCX-Dokumentation	2026-07-13 - OpenMower-TAF-ROS - Änderungsdocumentation - GPS-State Standardisierung - v0.1.docx